Do Language Models Plan Ahead for Future Tokens?

Wilson Wu
Department of Mathematics
University of Colorado Boulder
wiwu2390@colorado.edu

John X. Morris
Department of Computer Science
Cornell University
jxm3@cornell.edu

Lionel Levine
Department of Mathematics
Cornell University
levine@math.cornell.edu

arXiv:2404.00859v2 [cs.LG] 1 Aug 2024

Abstract

Do transformers "think ahead" during inference at a given position? It is known transformers prepare information in the hidden states of the forward pass at time step t that is then used in future forward passes $t + \tau$. We posit two explanations for this phenomenon: *pre-caching*, in which off-diagonal gradient terms present during training result in the model computing features at t irrelevant to the present inference task but useful for the future, and *breadcrumbs*, in which features most relevant to time step t are already the same as those that would most benefit inference at time $t + \tau$. We test these hypotheses by training language models without propagating gradients to past timesteps, a scheme we formalize as *myopic training*. In a constructed synthetic data setting, we find clear evidence for pre-caching. In the autoregressive language modeling setting, our experiments are more suggestive of the breadcrumbs hypothesis, though pre-caching increases with model scale.

1 Introduction

Humans are known to think ahead while speaking; decades of linguistics research (Huettig, 2015; Miller, 1951) have shown evidence that human language users internally predict upcoming language input, words and sometimes sentences ahead (Barthel et al., 2016).

Unlike humans, contemporary language models allocate a fixed amount of information processing for each token when "speaking" (Vaswani et al., 2017). Do language models, like humans, think ahead? Recent work (Pal et al., 2023; Hernandez et al., 2024) has shown that tokens beyond the immediate next token can be predicted by probing the hidden state of the language model. Model outputs at future tokens can be predicted to some extent using linear probes on model hidden states, and interventions on hidden states can predictably alter future outputs.[1]

These findings indicate that model activations at a given timestep are at least somewhat predictive of future outputs. However, it remains unclear why this might be: is this just a happenstance property of the data, or because the model is deliberately preparing information for future timesteps, at the expense of degrading performance on the current position?

We observe that gradients during training optimize weights for both the loss at the current token position as well as for tokens later in the sequence. We question to what extent current transformer weights dedicate resources to the current token vs. allocating it for future tokens.

[1]Code to reproduce our results can be found at https://github.com/wiwu2390/FutureGPT2-public

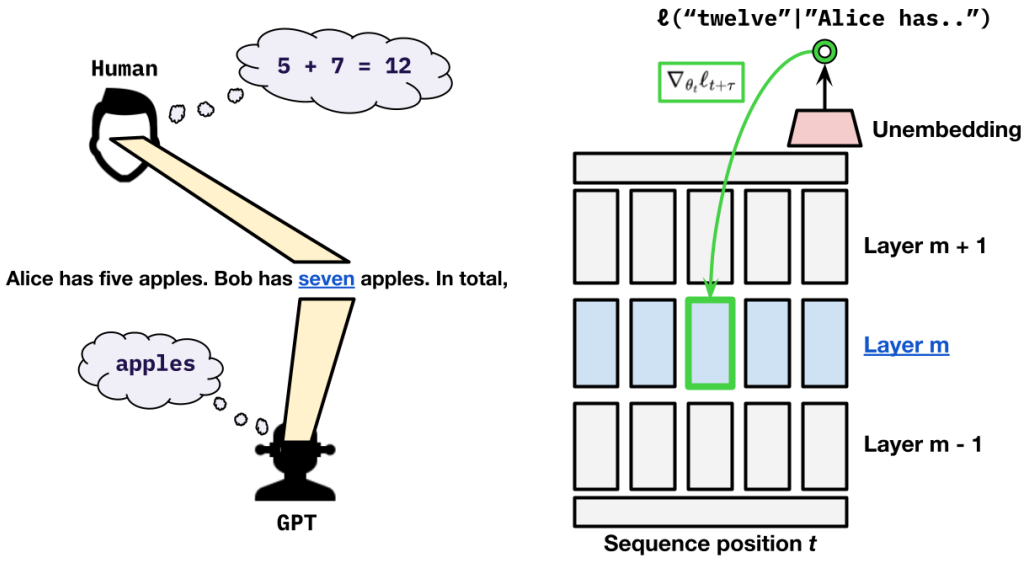


Figure 1: At which position is the computation required to correctly answer this math problem taking place? Cognitive science tells us that humans think ahead while speaking; we investigate the extent to which language models do the same.

We consider two possibilities: the *pre-caching* hypothesis, in which the transformer learns to compute features at time step t that are irrelevant to the inference task at that current time step but may be useful for future time steps $t + \tau$, and the *breadcrumbs* hypothesis, in which the features most relevant to time step t are already identical to those that would most benefit inference at time $t + \tau$. To evaluate which hypothesis might be correct, we propose a myopic training scheme that does not propagate gradients from the loss at the current position to hidden states from previous positions. We then evaluate the *myopia gap* in performance between myopically trained and vanilla transformers as a measure of pre-caching.

To consider whether language models might directly implement pre-caching, we design a synthetic scenario where the task can only be completed via explicit pre-caching. We configure a task where the model must precompute information for the next token, because otherwise the correct answer could not be accurately computed in a single forward pass. In this synthetic scenario, we find clear evidence that the transformer learns to pre-cache. When transformer-based sequence models must precompute information to minimize loss, they do so.

We then consider whether breadcrumbs or pre-caching is demonstrated in natural language models. Our experiments with myopic training suggest that, with small language models like GPT-2 (Radford et al., 2019), much less pre-caching occurs in this setting, pointing towards the breadcrumbs hypothesis. That is, we claim language models on this scale do not intentionally prepare information for the future to a significant extent. Instead, they compute features that are useful to predicting the immediate next token, which turn out to then be helpful at future steps; there is not a significant tradeoff between greedily optimizing for next token loss and ensuring future predictive performance.

However, we also find evidence that the importance of pre-caching increases with scale, becoming non-negligible with larger models, e.g. Pythia 2.8B (Biderman et al., 2023). This suggests that these larger models are "planning for the future" in a way that small models cannot.

2 Related work

Future token meta-prediction. Several recent works (nostalgebraist, 2020; Belrose et al., 2023; Pal et al., 2023; Cai et al., 2024) observe that transformer hidden states can be used to predict current and future tokens in a sequence, typically via linear probing. Notably, Hernandez et al. (2024) show that more complicated relationships are encoded linearly in hidden states, such as subject-object relations, implying that future tokens can also be predicted in specific cases. This future token predictivity has also been applied to speeding up inference by decoding future tokens in parallel (Stern et al., 2018; Cai et al., 2024). Unlike these works, we focus on the question of *how* the model learns to prepare hidden states that are useful for future prediction, possibly at the expense of current-token predictivity.

Probing. Our synthetic data experiments make use of *probing*, a technique where a simple auxiliary model is used to predict properties from target models' representations (Belinkov & Glass, 2019; Shi et al., 2016; Hewitt & Liang, 2019; Pimentel et al., 2020; Belinkov, 2021). Probing-based approaches are known to overestimate latent information if the classifier learns to do a task on its own (Belinkov, 2021), and probing analyses may only be informative when compared to probing a reasonable baseline (Hewitt & Liang, 2019). In our probing experiments, we avoid these pitfalls by ensuring that the function to be learned cannot possibly be computed by the probe itself.

Mechanistic interpretability. Our analysis of transformer models in a synthetic setting relates to the subfield of *mechanistic interpretability*, which seeks to understand models by isolating and explaining the behavior of their components (Olah et al., 2020; Bau et al., 2020; Meng et al., 2023; Nanda et al., 2023). Some of these works (Nanda et al., 2023; Li et al., 2023; Zhong et al., 2023) practice mechanistic interpretability by studying models trained on synthetic data. We apply some mechanistic interpretability techniques in a synthetic setting to study the problem of whether language models "think ahead" for future tokens. However, our approach also differs from that of mechanistic interpretability by analyzing the effect of the training procedure on the learned model.

3 Theory: Pre-caching or breadcrumbs?

Consider a generic causal sequence-to-sequence prediction task

$$(\mathbf{x}_1, \ldots, \mathbf{x}_n, \mathbf{y}_1, \ldots, \mathbf{y}_n) \sim \mathcal{D},$$

where $\mathcal{D}$ is a data distribution supported on $\mathbb{X}^n \times \mathbb{Y}^n$ for some domains $\mathbb{X}, \mathbb{Y}$. The task is to estimate the conditional expectations $\mathbb{E}_{\mathcal{D}}(\mathbf{y}_i \mid \mathbf{x}_1, \ldots, \mathbf{x}_i)$ for $1 \leq i \leq n$.[2] Note that we recover the autoregressive setting by setting $\mathbb{Y} = \mathbb{X}$ and $\mathbf{y}_i = \mathbf{x}_{i+1}$.

Transformer models trained on such tasks have been observed (Pal et al., 2023) to store information in hidden states during inference at position i that is then used in future inference at $j > i$. However, since the loss associated with each step i depends only on how well the model does at the immediate task of predicting $\mathbf{y}_i$, it may not be immediately obvious how this preparation for the future arises. We give names to two competing explanations:

- **Pre-caching:** The model "deliberately" computes and stores features that are expected to be useful for the future, even if they are irrelevant to the present.
- **Breadcrumbs:** The features that most benefit the present inference task are the same as those that are most useful to the future. When the model performs the present forward pass, it "unintentionally" leaves a trace ("breadcrumbs") that is then picked up by future passes.

[2]For classification tasks, we are typically interested in the conditional probabilities $\Pr_{\mathcal{D}}(\mathbf{y}_n = c \mid \mathbf{x}_1, \ldots, \mathbf{x}_n)$ for each class c. However, this can be subsumed into the generic case by letting $\mathbb{Y}$ be the probability simplex over all classes.

To disentangle these two explanations, we introduce a notion of *myopic* transformer models, which we show to be incapable of deliberate pre-caching—for these models, the extent to which past features are beneficial to the future is decided purely by the breadcrumbs explanation. Thus, the gap between vanilla and myopic transformer models is a quantitative measure of how much pre-caching is taking place.

3.1 Causal sequence modeling

Suppose, for the sake of exposition, that the transformer model G uses independent parameters for each position. [3] Let p be the parameter count of each forward pass of G. Then, letting $\boldsymbol{\theta}_i \in \Theta = \mathbb{R}^p$ be all parameters used by G at position i, a transformer G is a parameterized function

$$G\colon \mathbb{X}^n \times \Theta^n \to \mathbb{Y}^n, \qquad (\mathbf{x}_1, \ldots, \mathbf{x}_n; \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n) \mapsto (\hat{\mathbf{y}}_1, \ldots, \hat{\mathbf{y}}_n).$$

For $1 \leq i \leq n$, let $G_i(\mathbf{x}_1, \ldots, \mathbf{x}_n) \in \mathbb{Y}$ be the output of G's ith forward pass. Because of the causal masking within G, this depends only on $\mathbf{x}_1, \ldots, \mathbf{x}_i$ and $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_i$. That is, with slight abuse of notation, we may write

$$\hat{\mathbf{y}}_i = G_i(\mathbf{x}_1, \ldots, \mathbf{x}_n; \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n) = G_i(\mathbf{x}_1, \ldots, \mathbf{x}_i; \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_i).$$

3.2 Off-diagonal gradient terms

Now, letting $\mathcal{L}\colon \mathbb{Y} \times \mathbb{Y} \to \mathbb{R}_+$ be some choice of loss function, the expected loss ℓ of a transformer model with parameters $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n$ is

$$\ell(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n) := \mathbb{E}_{(\vec{\mathbf{x}}, \vec{\mathbf{y}}) \sim \mathcal{D}} \sum_{i=1}^{n} \mathcal{L}(G_i(\mathbf{x}_1, \ldots, \mathbf{x}_i; \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_i), \mathbf{y}_i) =: \sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_i),$$

the sum over $1 \leq i \leq n$ of the expected loss ℓ_i at position i. (We suppress the dependence on G and $\mathcal{D}$ for concision.) In practice, we always tie the weights across position. That is, all $\boldsymbol{\theta}_i$ are set equal to the same $\boldsymbol{\theta} \in \Theta$. Then, by the chain rule,

$$\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}, \ldots, \boldsymbol{\theta}) = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}_i} \ell(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n)\big|_{\boldsymbol{\theta}_1 = \ldots = \boldsymbol{\theta}_n = \boldsymbol{\theta}} = \sum_{i=1}^{n} \sum_{j=i}^{n} \nabla_{\boldsymbol{\theta}_i} \ell_j(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_j)\big|_{\boldsymbol{\theta}_1 = \ldots = \boldsymbol{\theta}_n = \boldsymbol{\theta}},$$

a sum over an upper-triangular expected Jacobian "matrix". The off-diagonal terms $i < j$, corresponding to the expected gradient of the model's future loss at position j with respect to its weights at position i, are the training signals that encourage pre-caching.

3.3 Measuring pre-caching: The myopia gap

We say a model is *myopic* when each forward pass G_i optimizes only ℓ_i without regard for future ℓ_j at $j > i$. In the untied weights case, the right definition is then apparent.

Definition 1. *The parameters $(\tilde{\boldsymbol{\theta}}_1, \ldots, \tilde{\boldsymbol{\theta}}_n) \in \Theta^n$ are **untied-myopic** if they satisfy*

$$\tilde{\boldsymbol{\theta}}_i \in \arg\min_{\boldsymbol{\theta}_i} \ell_i(\tilde{\boldsymbol{\theta}}_1, \ldots, \tilde{\boldsymbol{\theta}}_{i-1}, \boldsymbol{\theta}_i) \qquad \forall i \in \{1, \ldots, n\}. \tag{1}$$

Definition 2. *Let $\mathbb{M}$ be the feasible set of the constraints in Equation 1. The **untied myopia gap** is the smallest possible gap between the expected loss attained by a myopic model and the optimal model:*

$$p^* := \min_{(\tilde{\boldsymbol{\theta}}_1, \ldots, \tilde{\boldsymbol{\theta}}_n) \in \mathbb{M}} \ell(\tilde{\boldsymbol{\theta}}_1, \ldots, \tilde{\boldsymbol{\theta}}_n) - \min_{(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n) \in \Theta^n} \ell(\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n) \geq 0. \tag{2}$$

In the tied weights case, it is perhaps not immediately clear what the right definition of myopia should be. It does not suffice to simply constrain the minimizations in Equation 1 to $\tilde{\boldsymbol{\theta}}_1 = \ldots = \tilde{\boldsymbol{\theta}}_n$, since $\min_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}, \ldots, \boldsymbol{\theta})$ is optimizing for pre-caching (the dependence on arguments $j < i$) as well as the present inference (the dependence on argument i). Instead,

[3] For example, this is true of absolute position embedding weights.